UNIVERSITY OF SCIENCE

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

WEEKLY REPORT 2

Design Space Exploration of Shared L2 cache

Team lead: PhD. Le Minh Son

Mentor: Cao Van Nam

Full name: Huynh Ngoc Huy

Student ID: 24200087

Summary

What was done in this report:

- **Finalized the production parameters**: line 64 B, 16 ways, capacity 1 MiB (1024 sets), no L2 prefetcher, way shutdown per way (W = 1..16), L2 MSHR 32, 1 GHz, DDR4-2400, 22 nm.
- **Workloads**: used 4 workloads (loop, stream, temporal, graph), each with its footprint and a prediction of what happens when ways are shut down.
- **Replacement policy sweep** (LRU, Tree-PLRU, FIFO, NMRU, Random, LFSR): chose **Tree-PLRU**. Its miss rate / MPKI is close to LRU (temporal 11.13 / 0.516 vs 10.12 / 0.469; graph 40.92 / 3.835 vs 40.52 / 3.798) but it needs only 15 bits per set instead of 64.
- **Inclusion policy sweep**: chose **non-inclusive**. Exclusive gives almost the same miss rate, but L1 sends many more lines to L2 (e.g. temporal 18.9k vs 2.87M), and non-inclusive is simpler for way shutdown.
- **Way shutdown sweep** (4 cores, 1024 sets fixed, L2 leakage from McPAT):
  - loop: miss rate stays ~0.05% down to W13, then 23.7% at W12 and 100% at W ≤ 10 (16% slower). The limit is at W = 768 KiB / 64 KiB = 12, and W13 looks like the best point.
  - stream: miss rate is 100% at every W and run time does not change, while L2 leakage drops to 18.4% at W1.
  - Tree-PLRU gives results close to LRU at the measured W.
- **Checked the results against the predictions**: footprint matches the measurements, miss rate is monotonic in W, Tree-PLRU with 2 ways = LRU, misses are spread evenly over the 4 cores.
- **Next steps**: do not shut down ways when the working set is close to the remaining capacity; build a dynamic way shutdown algorithm and check it with gem5 and McPAT.

Finalizing production parameters: L2 capacity, replacement policy, inclusion policy

Because these are parameters of the product we are working on, we need to know them exactly before we can continue with the next steps, and the project description does not mention them. After asking for advice, we got the following parameters:

#	Parameter	Value	Reason
1	Line size	64 B	
2	Number of ways	16	
3	Capacity	1 MiB → 1024 sets, alias stride 64 KiB	The baseline team reported 1 MiB; L2 / total L1 = 1024 / 192 KiB ≈ 5.3
8	L2 prefetcher	No	To isolate the effect of the number of ways
9	Way shutdown granularity	Per way (W = 1..16)	Main points 16, 12, 8, 4, 2; extra points 14, 10, 6

#	Parameter	Value	Reason
10	L2 latency	Keep the baseline values	Does not affect miss rate
11	L2 MSHR	32, fixed	From Quyên
12	Clock, DRAM	1 GHz, DDR4-2400	Same as baseline
13	Node (only when energy is needed)	22 nm, 85 °C	CVA6 (Ariane) was taped out on GF 22FDX; CACTI supports 22 nm

- Why are there an i-cache and a d-cache?
  - Basically every CPU needs L1, because without L1 every instruction fetch and every data read goes straight to L2, and most of them are easy accesses (reused right away), so measuring the L2 miss rate would no longer mean anything. With L1, L2 only receives what L1 cannot keep -> this is the traffic that way shutdown affects.
  - The CPU fetches instructions and reads data in the same cycle, so MinorCPU has 2 separate ports: icache_port and dcache_port.

Workloads used

- Workloads used in this report, with predictions for way shutdown

Work-load	Algorithm	Purpose	Footprint (4 cores)	When ways are shut down
loop	Scan an array in order, repeat 100 times	A loop over the same data set	768 KiB	Sharp cliff at W12 (measured)
stream	c[i] = a[i] + k*b[i] on 3 large arrays	Streaming data pro-cessing, vector/DSP	6 MiB	Miss ~100% at every W
temporal	Random reads, 90% go to a small hot re-gion	Hot data that is used often	512 KiB hot + 4 MiB cold	Miss rises sharply when W ≤ 8
graph	Random walk on a graph by following pointers	Walking trees, graphs, linked lists	2 MiB	Miss rises gradually as W goes down

- CPU_WIDTH = 1: each stage can handle only 1 instruction per cycle

```
Cycle:  1      2       3       4       5       6       7
I1:    Fetch  Decode  Exec    Done
I2:           Fetch   Decode  Exec    Done
I3:                   Fetch   Decode  Exec    Done
I4:                           Fetch   Decode  Exec    Done
```

```
Cycle:    1       2       3      4       5      6      7
I1,I2:   Fetch   Decode  Exec   Done
I3,I4:           Fetch   Decode Exec    Done
```

  - Why 1: CVA6 in its default configuration is a single-issue core: it issues only 1 instruction per cycle. MinorCPU uses width 2 by default, so we lowered it to 1 to match CVA6.

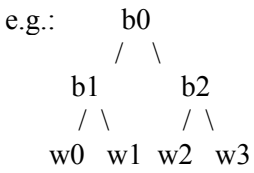
- WIDTH changes how fast the core sends requests down to L2 -> timing and order change.

- In fact, this simulation is not a multithreaded simulation, where one program runs and each core does one part of it; for that we would need to split the program into threads. Here we only simulate 4 cores starting the same loop at the same time and sharing L2, to study how instructions compete for space in L2 -> which gives the metrics we need to measure: miss rate, ...

Replacement policy sweep

- LRU (Least Recently Used): evicts the line that has not been used for the longest time
- Random (LFSR): picks a random line to evict
- Tree-PLRU (tree Pseudo-LRU): approximates LRU with few bits. True LRU needs many bits, and every hit must update many counters. Tree-PLRU replaces this with a binary tree of W − 1 bits (15 bits for 16 ways). Each bit remembers only one thing: "which half is older"
- NMRU (Not Most Recently Used): random, but never the line that was just used
- FIFO (first in first out): evicts the line that entered the cache first
- Actually, in each way = [valid | dirty | tag | 64 bytes of data], the tag only tells us which set the line is in, not the exact position of the line. So there is 1 extra field (one per set) that shows the exact position of the line in that set. Depending on the algorithm, the 'number of bits' is different. For example, LRU must store the exact order of all 16 ways, so each counter is 4 bits (0->15) and the number of bits = 16*4, while NMRU only stores the exact position of the most recently used line, so it needs only 4 bits instead of all 16 ways, so the number of bits = 4, ... and LRU works by searching a binary tree

e.g.:



- Miss rate = number of misses ÷ number of L2 accesses × 100, MPKI = number of misses ÷ number of instructions executed × 1000, that is, how many misses happen per 1000 instructions.
  - Miss rate tells which policy is better and by how many %, but MPKI tells whether that difference affects performance much or not. We can compute stall cycles per 1000 instructions = miss penalty (waiting time) * MPKI
  - We cannot use miss rate to evaluate DRAM traffic and energy, but we can use MPKI. Higher MPKI means more DRAM energy.
- Miss rate / MPKI table:

workload	LRU	TreePLRU	FIFO	NMRU	Random	LFSR
temporal	10.12 / 0.469	11.13 / 0.516	16.86 / 0.781	16.39 / 0.760	17.06 / 0.791	17.03 / 0.789
graph	40.52 / 3.798	40.92 / 3.835	42.81 / 4.013	42.52 / 3.987	42.66 / 4.000	42.70 / 4.005

workload	LRU	TreePLRU	FIFO	NMRU	Random	LFSR
stream	100	100	100	99.83	99.78	99.73
loop	0.01	0.01	0.01	0.01	0.01	0.01

-> We choose Tree-PLRU as the main replacement policy for this project because it has the best trade-off between miss rate and hardware. Instead of storing 64 bits for 16 ways, it needs only 15 bits. Its drawback is that when it chooses a line to evict, it chooses randomly - following the bits of the tree, which means it only approximates LRU and is not exact, but it is still better than NMRU - which picks any random line in the set, while PLRU rarely evicts a recently used line because PLRU is organized as a binary tree (draw the tree to see this). When the working set is larger than the number of ways, the miss rate is almost 100%. For example, a loop that keeps using the working set {A,B,C,D,E} on 4 ways: at first everything misses and 4 lines are loaded. Next, E is needed, so A is evicted and E is loaded. Then the loop repeats and needs A, so B is evicted and A is loaded, and so on, which leads to a 100% miss rate. In addition, the write needed on a hit also matters. For example, with LRU, when we evict 1 line we must go to that line, evict it, and then update 4 counters in 4 ways -> expensive. Tree-PLRU only needs to access 4 nodes to change the branches related to that line.

Inclusion policy sweep

- inclusive: every line in L1 must also be in L2 (on an L2 miss -> copy into both L2 and L1)
- non-inclusive: a line can be in both or in only one (on an L2 miss -> copy into both L2 and L1)
- exclusive: a line is in only one place (L1 or L2) (on an L2 miss -> only into L1, L2 does not keep it)
- Summary table

Type	Rule	On an L2 miss, the line from DRAM	When L1 evicts a line
Inclusive	Every line in L1 must be in L2	copied into both L2 and L1	no need to send it to L2 if the line is clean, because L2 already has it
Non-inclusive (NINE)	No rule: a line can be in both, or in only one	copied into both L2 and L1	sent to L2 only if the line is dirty
Exclusive	A line is in only one place: L1 or L2	only into L1, L2 does not keep it	sent to L2 even if it is clean. L2 works as a "store for lines evicted by L1" (victim cache)

- Data from gem5

workload	L2 miss% non-incl / excl	MPKI nonincl / excl	cycles excl/nonincl	L1->L2 writes nonincl / excl
graph	40.52 / 39.74	3.798 / 3.725 (-1.9%)	0.996	41k / 1.98M

workload	L2 miss% non-incl / excl	MPKI nonincl / excl	cycles excl/nonincl	L1->L2 writes nonincl / excl
temporal	10.12 / 10.09	0.469 / 0.468	1.000	18.9k / 2.87M
stream	100 / 100	5.342 / 5.341	1.000	370k / 894k
loop	0.01 / 0.01	~0	1.000	1.22M / 1.22M (dirty migration from the init loop)

- In gem5 there are actually only 2 options for this: mostly_incl (really non-inclusive) and mostly_excl (L1 turns on writeback_clean=true to send clean lines to L2, so when L1 evicts, L2 is where the lines go)
- About capacity:
  - incl & non-incl: effective capacity ≈ L2 capacity.
  - excl: = (total L1) + L2
- For this project we propose non-inclusive, because non-incl is much simpler: in way shutdown, clean lines are dropped and dirty lines are written to DRAM, and many lines are still in D1 so less is lost && comparing with exclusive across the workloads, the numbers are not very different, but in L1->L2 writes exclusive is much higher in how many times L1 sends lines to L2, e.g. stream 370k/894k, and for a project that cares about energy like this one, excl has almost no chance, since we are studying way shutdown to reduce leakage power, but excl, although it adds ways, also increases energy at the same time, so the two cancel out -> no benefit
- And why excl has so many writes: when L1 evicts a line, even a clean one, it must be written back to L2, because L2 deleted its copy when it moved the line up to L1. So each L2 hit becomes a round trip L2 → L1 → L2, which costs one more L2 write -> the number of writes grows a lot

Way shutdown sweep

- In this report we study way shutdown with a fixed number of L2 sets, to see whether it really reduces leakage power.
- Results of the sweep with 4 cores, LRU, non-incl, L2 1 MiB, loop

W	C(W)	L2 miss %	MPKI	L1D miss lat (cyc)	cycles/W16	L2 leakage (mJ)
16	1024K	0.04	0.002	12.0	1.000	13.84 (100%)
14	896K	0.05	0.002	12.0	1.000	---
13	832K	0.05	0.002	12.0	1.000	---
12	768K	23.7	0.989	24.2	1.040	---
11	704K	80.0	3.333	51.5	1.128	---
10	640K	100	4.169	61.3	1.159	---
8	512K	100	4.169	61.3	1.160	8.78 (63.4%)
6	384K	100	4.169	61.3	1.159	---
4	256K	100	4.169	61.2	1.160	5.38 (38.9%)

W	C(W)	L2 miss %	MPKI	L1D miss lat (cyc)	cycles/W16	L2 leakage (mJ)
2	128K	100	4.169	61.2	1.159	3.85 (27.8%)
1	64K	100	4.169	61.3	1.159	2.95 (21.3%)

- Results of the sweep with 4 cores, LRU, non-incl, L2 1 MiB, stream

W	C(W)	L2 miss %	MPKI	L1D miss lat (cyc)	cycles/W16	L2 leakage (mJ)
16	1024K	100	5.209	96.7	1.000	10.07 (100%)
8	512K	100	5.210	96.8	1.000	5.51 (54.7%)
4	256K	100	5.209	96.6	1.000	3.38 (33.6%)
2	128K	100	5.210	96.6	0.998	2.41 (23.9%)
1	64K	100	5.210	96.3	0.997	1.85 (18.4%)

- Results with the same configuration, but with Tree-PLRU replacement, loop

W	C(W)	L2 miss %	MPKI	L1D miss lat (cyc)	cycles/W16	L2 leakage (mJ)
16	1024K	0.04	0.002	12.0	1.000	13.84 (100%)
8	512K	97.86	4.080	60.5	1.158	8.77 (63.4%)
4	256K	100	4.169	61.2	1.160	5.38 (38.9%)
2	128K	100	4.169	61.2	1.159	3.85 (27.8%)
1	64K	---	---	---	---	---

- Note:
  - Some energy cells in the table are empty because McPAT only runs with W = 2^n, so 14, 12, 10... cannot be run with McPAT; if needed, we can use interpolation from the power values we already have, as an approximation
  - McPAT does not model the replacement state bits, so LRU and Tree-PLRU have the same leakage power; the difference comes only from run time

Result evaluation

- From the LRU sweep, we see that from W13 onward the miss rate starts to rise sharply (0.05 -> 23.7), and at W10 it fully thrashes. The reason is that the total footprint is 4 cores x 192KB = 768KB, and 1 way holds 64KB, so the limit is at W = 768/64 = 12. The loop scans 12 lines per set in a cycle. When W < 12, LRU always evicts exactly the line that will be needed next -> 100% miss. But at W12, which is exactly equal to the footprint, MR is still 23.7: besides the data array, L2 also has to hold the code, stack and other variables of the 4 cores, and we cannot make sure the lines spread perfectly evenly over 1024 sets, so some sets have more than 12 lines and thrash -> we need to keep at least 1 extra way.

- W13 seems to be the best point: the miss rate is about the same as W14 and W16, but leakage power can drop (we do not have real data, but we can infer it). This is because at W13 the footprint is 832KB > 768KB -> the 3 ways that are shut down hold no data that is in use
- With Tree-PLRU, at every W the results are not much different from LRU in any metric -> it can be used to cut hardware cost, so the proposal to use PLRU in the replacement section is correct
- For the stream workload, each array = 512KB, each core = 3 * 512 = 1.5MB -> footprint = 4 cores * 1.5 = 6MB -> 6MB/64KB = 96 lines/set >> 16. It scans in order and each line is used once -> MR = 100% at every W

Checking the results against the predictions at the start, for 2 workloads: loop and stream

1. **Footprint matches the measurements**: loop at W10 has 287 058 L2 accesses per core = 3 072 lines × 93.4 passes (100 passes minus the part inside warmup)
2. **Monotonic in W**: with LRU and a fixed number of sets, the miss rate does not go up when W goes up. The loop table holds at every point
3. **Tree-PLRU with 2 ways = LRU**: a 2-way tree has only 1 bit, which is true LRU. The W2 numbers of the two policies are the same
4. **Misses are spread evenly over the 4 cores**: loop W10 has ~287k misses per core, which matches 4 cores running the same program

-> we can fully predict the miss rate from the configuration data

- Proposed improvements for the next report:
  - Do not shut down ways when the working set is close to the remaining capacity. As we saw, when the working set is a bit larger than the number of active ways (loop at W8), Tree-PLRU also drops to 100% miss like LRU, while LFSR is only about 60%. The fix is to use MPKI itself as a signal: count misses in every interval of N instructions, and if MPKI goes above a threshold, turn ways back on. In this way, MPKI is both the metric for choosing the policy and the control signal for way shutdown at run time
  - A dynamic way shutdown algorithm that reduces leakage power while keeping the miss rate as low as possible. Keep reading and studying existing work, and check again with gem5 and McPAT to find the best and most suitable approach